ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO THỰC HÀNH

LAB 02: LẬP TRÌNH ASSEMBLY CƠ BẢN

Basic Assembly Programming – Lập trình hợp ngữ x86

Nhóm: Nhóm 4
Sinh viên 1: Bùi Thái Sơn – 25521588
Sinh viên 2: Võ Quốc Thịnh – 25521787
Sinh viên 3: Lê Thành Đạt – 25520270

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Chi tiết các bài tập và giải thích mã nguồn **1**
1.1 Bài 2.1 1
1.2 Bài 2.2 1
1.3 Bài 2.3 2
1.4 Bài 2.4 4
1.5 Bài 2.5 5
1.6 Bài 2.6 7

2 Minh chứng kết quả thực thi **11**

1 Chi tiết các bài tập và giải thích mã nguồn

1.1 Bài 2.1

Giải thích: Ta tính độ dài chuỗi `msg`, lưu ở biến `msg_len`. Cộng thêm `0x80` để chuyển sang ASCII. Khi khởi chạy, chương trình chỉ cần nạp đúng địa chỉ buffer và output chuỗi.

Mã nguồn chương trình (`bai2.1.s`):

```
.section .data
msg:
    .string "IloveCR7"
msg_len = . - msg - 1 # Lay dia chi hien tai - dia chi msg - 1 (tru byte null)

output:
    .byte msg_len + 0x30 # Cong 48 (0x30) de chuyen thanh ky tu ASCII
    .byte 0x0a           # Ky tu xuong dong '\n'

.section .text
    .globl _start
_start:
    movl $output, %ecx   # Dia chi buffer can xuat
    movl $0x2, %edx      # Do dai xuat ra: 2 bytes
    movl $0x1, %ebx      # File descriptor: stdout (1)
    movl $0x4, %eax      # System call number: sys_write (4)
    int $0x80            # Goi kernel xuat chuoi

    movl $0x1, %eax      # System call number: sys_exit (1)
    int $0x80            # Thoat chuong trinh
```

1.2 Bài 2.2

Giải thích: Nhập lần lượt 4 số. Với mỗi số, ta nhận kí tự hàng chục, chuyển sang số nguyên, nhân 10; nhận kí tự hàng đơn vị, chuyển sang số nguyên; lưu tổng lại ta được số tương ứng. Sau đó, ta cộng 4 số đã được nhập vào rồi tiến hành nạp tham số vào các thanh ghi để chia cho 8 lấy số dư, chuyển kết quả sang mã ASCII rồi in ra màn hình.

Mã nguồn chương trình (`bai2_2.s`):

```
.section .data
nhapvao:
    .string  "Enter a number (2-digit): "
kq:
    .string "Remainder: "
newline:
    .byte 10
ans:
    .int 0

.section .bss
    .lcomm number1, 4

.section .text
    .globl _start

jmp _start

nhapso:
    movl $4, %eax        # sys_write in loi nhac
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $26, %edx
    int $0x80

    movl $3, %eax        # sys_read doc 3 bytes tu STDIN
    movl $0, %ebx
    movl $number1, %ecx
    movl $3, %edx
    int $0x80
```

```
31
32      movzbl number1, %eax    # Doc chu so hang chuc
33      subl $48, %eax          # Chuyen thanh gia tri nguyen
34      imull $10, %eax         # Nhan voi 10
35
36      movzbl number1+1, %ebx  # Doc chu so hang don vi
37      subl $48, %ebx
38      addl %ebx, %eax         # Tong gia tri cua so vua nhap
39
40      addl %eax, ans          # Cong don vao bien tong ans
41      ret
42
43  inkq:
44      movl ans, %eax       # Nap tong vao %eax
45      xorl %edx, %edx      # Xoa %edx de chuan bi chia 64-bit
46      movl $8, %ecx        # So chia = 8
47      divl %ecx            # Chia: thuong o %eax, so du o %edx
48
49      movl %edx, ans
50      addl $48, ans        # Chuyen so du sang ky tu ASCII
51
52      movl $4, %eax        # In chuoi thong bao "Remainder: "
53      movl $1, %ebx
54      movl $kq, %ecx
55      movl $11, %edx
56      int $0x80
57
58      movl $4, %eax        # In ky tu so du
59      movl $1, %ebx
60      movl $ans, %ecx
61      movl $1, %edx
62      int $0x80
63
64      movl $4, %eax        # In dau xuong dong
65      movl $1, %ebx
66      movl $newline, %ecx
67      movl $1, %edx
68      int $0x80
69      ret
70
71  _start:
72      call nhapso
73      call nhapso
74      call nhapso
75      call nhapso
76      call inkq
77
78      mov $1, %eax         # sys_exit
79      int $0x80
```

1.3 Bài 2.3

Giải thích: Ta nhập chuỗi và số bước dịch n (từ 0 đến 9). Chương trình duyệt qua từng ký tự: nếu là chữ in hoa hoặc chữ in thường, ta dịch đi n đơn vị và kiểm tra xoay vòng bằng cách trừ đi 26 nếu vượt quá bảng chữ cái; các ký tự còn lại (số, khoảng trắng, ký tự đặc biệt) được giữ nguyên. Cuối cùng, ta xuất chuỗi đã được mã hóa ra màn hình.

Mã nguồn chương trình (`bai2.3.s`):

```
1   .section .rodata
2   enter_string:
3       .ascii "Enter a string (<255 chars): "
4   enter_string_len = .-enter_string
5
6   shift_string:
7       .ascii "Enter n (0-9): "
8   shift_string_len = .-shift_string
9
10  .section .bss
11      .lcomm input_str, 256
```

```
    .lcomm input_len, 4
    .lcomm n, 2

.section .text
    .globl _start

printf:
    movl $0x1, %ebx         # stdout
    movl $0x4, %eax         # write
    int $0x80
    ret

read:
    xorl %ebx, %ebx         # stdin
    movl $0x3, %eax         # read
    int $0x80
    ret

cipher:
    movl $0, %ecx
    movl $input_str, %edx
    decl %edx

.cipher_loop:
    cmpl input_len, %ecx
    je .fin
    incl %edx
    incl %ecx
    cmpb $'A', (%edx)
    jb .cipher_loop     # Ky tu ngoai khoang (giu nguyen)
    cmpb $'Z', (%edx)
    movb $'A', %bl
    jbe .shift          # Chu hoa ('A' <= x <= 'Z')

    cmpb $'a', (%edx)
    jb .cipher_loop     # Ky tu dac biet nam giua 'Z' va 'a'
    cmpb $'z', (%edx)
    movb $'a', %bl
    jbe .shift          # Chu thuong ('a' <= x <= 'z')

    jmp .cipher_loop    # Ky tu lon hon 'z'

.fin:
    ret

.shift:
    movb (%edx), %al
    subb %bl, %al       # Dua ve khoang 0 .. 25
    addb n, %al         # Cong buoc dich n
    cmpb $26, %al       # Kiem tra tran 26
    jb .done
    subb $26, %al       # Xoay vong bang cach tru 26

.done:
    addb %bl, %al       # Dua ve lai ma ASCII goc
    movb %al, (%edx)    # Ghi de ky tu da ma hoa vao buffer
    jmp .cipher_loop

_start:
    movl $enter_string, %ecx
    movl $enter_string_len, %edx
    call printf

    movl $input_str, %ecx
    movl $256, %edx
    call read
    movl %eax, input_len

    movl $shift_string, %ecx
    movl $shift_string_len, %edx
    call printf

    movl $n, %ecx
```

```
    movl $0x2, %edx
    call read
    subb $0x30, n
    call cipher

    movl $input_str, %ecx
    movl input_len, %edx
    call printf

    movl $1, %eax
    int $0x80
```

1.4 Bài 2.4

Giải thích: Thay vì duyệt từng số từ x đến y, ta áp dụng công thức $\lfloor y/10 \rfloor - \lfloor (x-1)/10 \rfloor$ để đếm trực tiếp số lượng bội của 10 trong đoạn $[x, y]$. Sau khi nhập hai số x và y, chương trình tính thương số tương ứng của từng số khi chia cho 10 rồi lấy hiệu của chúng, chuyển kết quả sang mã ASCII và đẩy vào ngăn xếp để in ra màn hình.

Mã nguồn chương trình (`bai2.4.s`):

```
.section .data
hihi:
    .string "Nhap so: "
string_len = . - hihi
newline:
    .ascii "\n"

.section .bss
    .lcomm x, 3
    .lcomm y, 3

.section .text
.globl _start

_start:
    # 1. Nhap va tinh [(x - 1) / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80

    movl $0, %ebx
    movl $3, %eax
    movl $x, %ecx
    movl $3, %edx
    int $0x80

    movzbl x, %edi
    subl $'0', %edi
    imull $10, %edi

    movzbl x+1, %eax
    subl $'0', %eax
    addl %eax, %edi

    decl %edi            # Tinh x - 1
    movl $10, %ecx
    movl %edi, %eax
    xorl %edx, %edx
    divl %ecx            # Chia 10: thuong nam o %eax
    movl %eax, %edi      # Luu [(x - 1) / 10] vao %edi

    # 2. Nhap va tinh [y / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80
```

```
50
51      movl $0, %ebx
52      movl $3, %eax
53      movl $y, %ecx
54      movl $3, %edx
55      int $0x80
56
57      movzwl y, %eax
58      subb $'0', %al
59      subb $'0', %ah
60
61      movzbl %al, %ecx
62      imull $10, %ecx
63      movzbl %ah, %edx
64      addl %edx, %ecx      # y gio nam o %ecx
65
66      movl %ecx, %eax
67      movl $10, %ecx
68      xorl %edx, %edx
69      divl %ecx            # Chia 10: thuong o %eax
70      movl %eax, %esi      # Luu [y / 10] vao %esi
71
72      # 3. Tinh hieu so va xuat ket qua
73      subl %edi, %esi      # %esi = [y/10] - [(x-1)/10]
74      addl $'0', %esi      # Chuyen thanh ky tu ASCII
75      pushl %esi           # Day vao dinh ngan xep
76
77      movl $4, %eax
78      movl $1, %ebx
79      movl %esp, %ecx      # Dung dia chi %esp lam buffer xuat
80      movl $1, %edx
81      int $0x80
82
83      movl $4, %eax        # Xuat dau xuong dong
84      movl $1, %ebx
85      movl $newline, %ecx
86      movl $1, %edx
87      int $0x80
88
89      movl $1, %eax        # Thoat
90      xorl %ebx, %ebx
91      int $0x80
```

1.5 Bài 2.5

Giải thích: Ta nhập năm sinh 4 chữ số, chuyển thành số nguyên rồi lấy mốc năm 2026 trừ đi để tính tuổi. Nếu tuổi nằm trong khoảng từ 16 đến 30 thì in ra `"Thuoc Doan TN"`, ngược lại in `"Khong thuoc Doan TN"`. Tiếp theo, ta chia liên tiếp số tuổi cho 10 rồi dùng ngăn xếp (stack) để in ra số tuổi cụ thể theo đúng thứ tự các chữ số.

Mã nguồn chương trình (`bai2.5.s`):

```
1  .section .data
2  hello:
3      .ascii "Nhap nam sinh: "
4  string_len = . - hello
5
6  Thuoc:
7      .ascii "Thuoc Doan TN\n"
8  Thuoc_len = . - Thuoc
9
10 Khongthuoc:
11     .ascii "Khong thuoc Doan TN\n"
12 Khongthuoc_len = . - Khongthuoc
13
14 intuoi:
15     .ascii "Tuoi: "
16 intuoi_len = . - intuoi
17
18 newline:
```

```
    .ascii "\n"

.section .bss
    .lcomm tuoi, 5
    .lcomm out, 1

.section .text
.globl _start

_start:
    # 1. Nhap nam sinh tu ban phim
    movl $4, %eax
    movl $1, %ebx
    movl $hello, %ecx
    movl $string_len, %edx
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $tuoi, %ecx
    movl $5, %edx
    int $0x80

    # 2. Chuyen 4 ky tu thanh so nguyen trong %edi
    xorl %edi, %edi

    movzbl tuoi, %eax
    subl $'0', %eax
    imull $1000, %eax
    addl %eax, %edi

    movzbl tuoi+1, %eax
    subl $'0', %eax
    imull $100, %eax
    addl %eax, %edi

    movzbl tuoi+2, %eax
    subl $'0', %eax
    imull $10, %eax
    addl %eax, %edi

    movzbl tuoi+3, %eax
    subl $'0', %eax
    addl %eax, %edi

    # 3. Tinh tuoi: 2026 - nam_sinh
    movl $2026, %eax
    subl %edi, %eax
    movl %eax, %edi

    # 4. Kiem tra khoang [16, 30]
    cmpl $16, %edi
    jl ngoai_khoang
    cmpl $30, %edi
    jg ngoai_khoang

    movl $4, %eax
    movl $1, %ebx
    movl $Thuoc, %ecx
    movl $Thuoc_len, %edx
    int $0x80
    jmp nang_cao

ngoai_khoang:
    movl $4, %eax
    movl $1, %ebx
    movl $Khongthuoc, %ecx
    movl $Khongthuoc_len, %edx
    int $0x80

nang_cao:
    movl $4, %eax
    movl $1, %ebx
```

```
    movl $intuoi, %ecx
    movl $intuoi_len, %edx
    int $0x80
    call print

done:
    movl $1, %eax
    xorl %ebx, %ebx
    int  $0x80

# Thu tuc print: in so nguyen trong %edi ra man hinh
print:
    movl %edi, %eax
    xorl %esi, %esi

chia_loop:
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx            # Chia 10 lay so du vao %edx
    addb $'0', %dl       # Chuyen sang ma ASCII
    pushl %edx           # Day vao ngan xep
    incl %esi            # Tang bien dem chu so
    cmpl $0, %eax
    jne chia_loop

in_loop:
    popl %eax            # Rut chu so theo thu tu dung
    movb %al, out
    movl $4, %eax
    movl $1, %ebx
    movl $out, %ecx
    movl $1, %edx
    int $0x80
    decl %esi
    jnz in_loop

    movl $4, %eax        # In ky tu xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret
```

1.6 Bài 2.6

Giải thích: Ta nhập năm 4 chữ số và kiểm tra năm nhuận theo lịch Gregory: nếu là năm tròn thế kỷ (kết thúc bằng `00`) thì kiểm tra chia hết cho 400; các năm còn lại chỉ cần chia hết cho 4. Sau khi in kết quả `"Nam nhuan"` hoặc `"Nam khong nhuan"`, chương trình liên tục tăng năm lên 1 và kiểm tra cho đến khi tìm được năm nhuận kế tiếp rồi in ra màn hình.

Mã nguồn chương trình (`bai2_6.s`):

```
.section .data
nhapvao:
    .string  "Enter a year (4-digit): "
kqnamnhuan:
    .string "Nam nhuan\n"
kqnamkhongnhuan:
    .string "Nam khong nhuan\n"
namnhuantieptheouwu:
    .string "Nam nhuan ke tiep: "
newline:
    .byte 10
ans:
    .int 0

.section .bss
    .lcomm number1, 5
    buffer_num: .space 16

```

```
.section .text
    .globl _start

jmp _start

namnhuantieptheo:
    addl $1, ans         # Tang nam len 1
    movl ans, %eax
    xorl %edx, %edx
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Khong chia het cho 4 -> lap tiep

    movl ans, %eax
    xorl %edx, %edx
    movl $100, %ecx
    divl %ecx
    cmpl $0, %edx
    jne okroido          # Chia het cho 4 va khong chia het cho 100 -> Nam nhuan!

    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Chia het cho 100 nhung khong chia het cho 400 -> Lap tiep

okroido:
    movl $4, %eax
    movl $1, %ebx
    movl $namnhuantieptheouwu, %ecx
    movl $19, %edx
    int $0x80

    movl ans, %eax
    movl $buffer_num + 10, %esi
    movl $0, %ecx

chuyen_doi:
    decl %esi
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx
    addb $48, %dl
    movb %dl, (%esi)
    incl %ecx
    cmpl $0, %eax
    jne chuyen_doi

    movl $4, %eax
    movl $1, %ebx
    movl %esi, %ecx
    movl $buffer_num + 10, %edx
    subl %esi, %edx
    int $0x80

    call xuonghang
    ret

xuonghang:
    movl $4, %eax
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret

_start:
    movl $4, %eax
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $23, %edx
```

```
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $number1, %ecx
    movl $5, %edx
    int $0x80

    # Chuyen 4 ky tu thanh so nguyen ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans

    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans

    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    # Kiem tra 2 chu so cuoi co cung bang 0 hay khong
    movl $0, %ecx
    addl %eax, %ecx
    addl %ebx, %ecx
    cmpl $0, %ecx
    je tieptuckiemtra
    jmp saudo

tieptuckiemtra:          # Nam the ky: kiem tra chia het cho 400
    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namkhongnhuan

lanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

namkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80
    jmp end

saudo:                   # Nam binh thuong: kiem tra 2 chu so cuoi chia het cho 4
    movl $0, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    movl ans, %eax
    xorl %edx, %edx
```

```
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne tnamkhongnhuan

tlanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

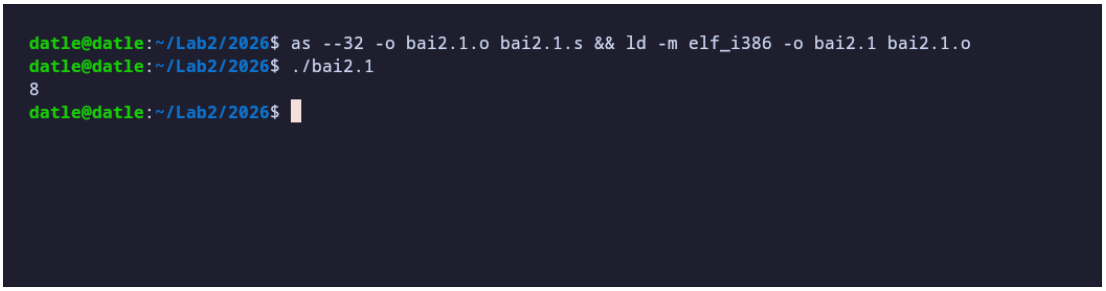
tnamkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80

end:
    # Tinh lai gia tri nam goc va tim nam nhuan tiep theo
    movl $0, ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans
    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans
    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

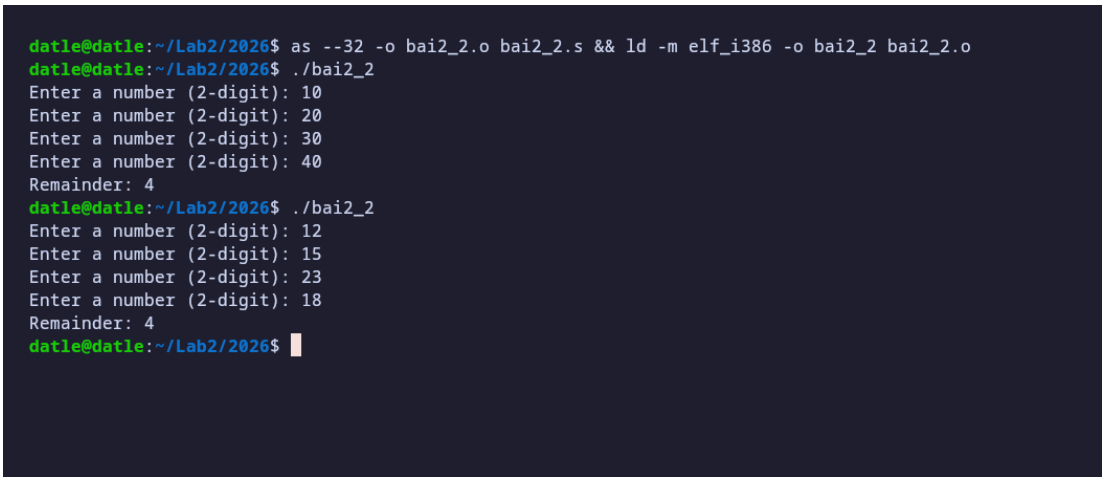
    call namnhuantieptheo

    mov $1, %eax
    int $0x80
```

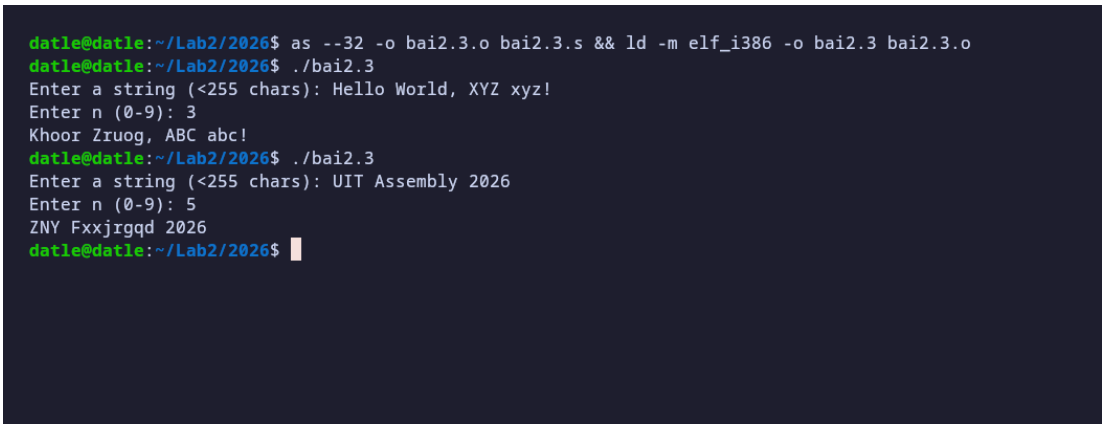
2 Minh chứng kết quả thực thi



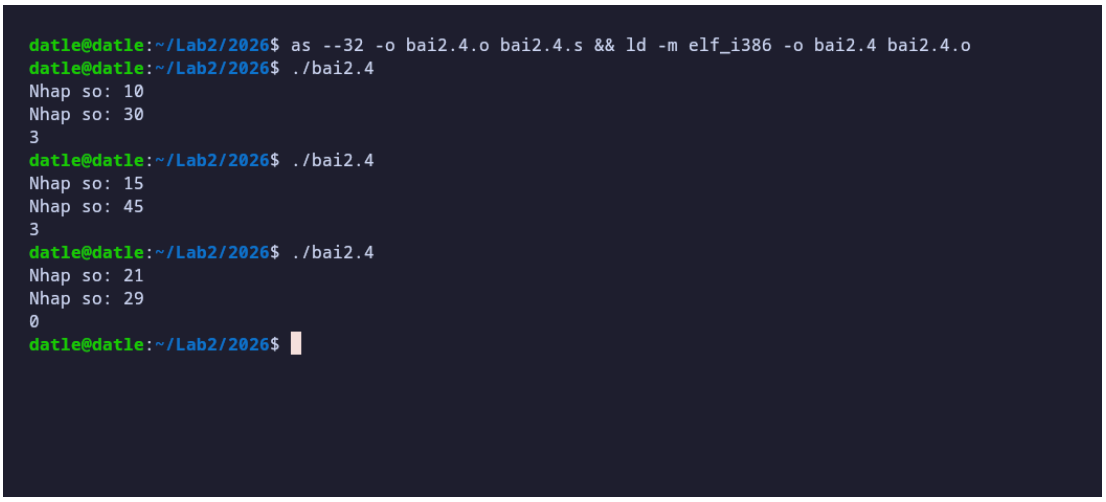
Hình 1: Kết quả biên dịch, liên kết và thực thi Bài 2.1 (In độ dài chuỗi cho trước).



Hình 2: Kết quả biên dịch, liên kết và thực thi Bài 2.2 (Số dư của tổng 4 số khi chia 8).



Hình 3: Kết quả biên dịch, liên kết và thực thi Bài 2.3 (Mã hóa Caesar nâng cao 1 & 2).



Hình 4: Kết quả biên dịch, liên kết và thực thi Bài 2.4 (Đếm số lượng bội của 10 trong đoạn $[x, y]$).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2.5.o bai2.5.s && ld -m elf_i386 -o bai2.5 bai2.5.o
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2005
Thuoc Doan TN
Tuoi: 21
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 1990
Khong thuoc Doan TN
Tuoi: 36
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2015
Khong thuoc Doan TN
Tuoi: 11
datle@datle:~/Lab2/2026$
```

Hình 5: Kết quả biên dịch, liên kết và thực thi Bài 2.5 (Kiểm tra tuổi đoàn và in kèm số tuổi).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2_6.o bai2_6.s && ld -m elf_i386 -o bai2_6 bai2_6.o
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2024
Nam nhuan
Nam nhuan ke tiep: 2028
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2023
Nam khong nhuan
Nam nhuan ke tiep: 2024
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 1900
Nam khong nhuan
Nam nhuan ke tiep: 1904
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2000
Nam nhuan
Nam nhuan ke tiep: 2004
datle@datle:~/Lab2/2026$
```

Hình 6: Kết quả biên dịch, liên kết và thực thi Bài 2.6 (Kiểm tra năm nhuận và tìm năm nhuận kế tiếp).